MODULE · PYTHON FUNDAMENTALS

DAY 4 — PYTHON FUNDAMENTALS

Functions • Built-in vs User-Defined • Arguments • Return • Scope • Building Reusable Code

Module	Refresh Topics	Practice	Task
Python	Functions, Arguments, Return, Scope	20 function questions	ATM Menu

What You'll Learn Today

- What a function is, why we break code into reusable pieces, and the DRY principle
- The difference between **built-in functions** (already provided by Python) and **user-defined functions** (that you create)
- How to define and call a function using def
- Passing information into functions using arguments and parameters
- The difference between positional, default, and keyword arguments
- Sending data back with return, and how it differs from print
- Local vs global variable scope, and why the global keyword is needed
- Building a practical ATM Menu program using functions

1 FUNCTIONS — Reusable Blocks of Code

What is a Function?

A function is a named block of code that performs a specific task. Instead of writing the same lines of code again and again, you define the logic once inside a function and simply call it by name whenever you need it. Functions make programs shorter, easier to read, and easier to fix. This idea is often called the **DRY principle — Don't Repeat Yourself**: if you notice yourself copy-pasting the same lines more than once, that logic is a good candidate to become a function.

Why Use Functions?

- **Reusability** — write the logic once, call it as many times as you need, with different data each time
- **Readability** — a well-named function (like calculate_total()) explains what the code does without reading every line
- **Easier debugging** — if something goes wrong, you only need to check inside the one function responsible
- **Abstraction** — the caller doesn't need to know *how* the function works internally, only *what* it does
- **Organization** — large programs become manageable when broken into small, focused functions

Anatomy of a Function

Every function definition is built from the same fixed parts. Understanding each part makes it much easier to read and write any function you come across:

Part	Purpose
`def`	Keyword that tells Python "a function definition starts here"
Function name	How you will call the function later; follows variable naming rules
Parentheses `()`	Hold the parameters (inputs) the function accepts — can be empty
Colon `:`	Marks the start of the function body, same as with `if` and `for`
Indented body	The block of code that runs each time the function is called
`return` (optional)	Sends a value back to the caller; if omitted, the function returns `None`

Syntax

CODE

```
def function_name():
    # code that runs when the function is called
```

Example 1 — A Simple Function

CODE

```
# Define a function
def greet():
    print("Hello, welcome!")

# Call the function
greet()
greet()
```

OUTPUT

```
Hello, welcome!
Hello, welcome!
```

Example 2 — Function Defined Before It Is Used

CODE

```
def square(n):
    print(n * n)

square(4)
square(7)
```

OUTPUT

```
16
49
```

Key Points About Functions

- A function only runs when it is called — defining it does nothing by itself
- Function names follow the same rules as variable names (lowercase, underscores, no spaces)
- The colon `:` after the parentheses is required, just like with `if` and `for`
- Code inside the function must be indented (4 spaces)
- A function must be defined before it is called in the program
- A function can call other functions, and can even call itself (this is called recursion — a topic for later)

2 BUILT-IN FUNCTIONS vs USER-DEFINED FUNCTIONS

Built-in Functions

Built-in functions are functions that **already exist inside Python** — you do not need to define them or import anything to use them. Python ships with dozens of these ready-made tools for the most common tasks: displaying output, taking input, measuring length, converting data types, and performing quick calculations. You have already been using several of these since Day 1 without formally naming them.

Built-in Function	What It Does	Example
`print()`	Displays output on the screen	`print("Hi")`
`input()`	Takes text input from the user	`input("Name: ")`
`len()`	Returns the number of items/characters	`len("hello")` → 5
`type()`	Returns the data type of a value	`type(5)` → int
`int() / float() / str()`	Convert a value to that data type	`int("10")` → 10
`range()`	Generates a sequence of numbers	`range(1, 5)`
`sum()`	Adds up all items in a list	`sum([1,2,3])` → 6
`min() / max()`	Returns smallest / largest value	`max(4, 9, 2)` → 9
`sorted()`	Returns a sorted version of a list	`sorted([3,1,2])`
`abs() / round()`	Absolute value / rounds a number	`round(3.7)` → 4

CODE

```
# A few built-in functions in action
numbers = [4, 9, 2, 7]

print(len(numbers))     # count of items
print(max(numbers))     # largest value
print(sum(numbers))     # total of all items
print(sorted(numbers))  # sorted list
```

OUTPUT

```
4
9
22
[2, 4, 7, 9]
```

User-Defined Functions

User-defined functions are functions that **you write yourself** using the `def` keyword, to perform a task specific to your own program. Python has no way of knowing in advance how *your* ATM program should calculate a balance or how *your* game should check a winner — so you define that logic yourself. Every function you wrote in Section 1 (`greet()`, `square()`) is a user-defined function.

Built-in Function

Already written and provided by Python itself. Ready to use immediately, works the same in every Python program, and cannot be changed.

User-Defined Function

Written by the programmer for a specific need. Must be defined with `def` before it can be called, and can be edited freely.

Aspect	Built-in Function	User-Defined Function
Origin	Comes pre-built with Python	Written by the programmer
Needs `def`?	No	Yes, always
Can be modified?	No	Yes, fully customizable
Example	`len()`, `print()`	`greet()`, `check_balance()`

⚠ **Note:** You can freely mix both kinds — a user-defined function almost always calls built-in functions inside it, exactly like `greet()` calls the built-in `print()` internally.

3 ARGUMENTS — Passing Data Into Functions

Arguments let you send information into a function so it can work with different values each time it runs, instead of always doing the exact same thing.

Parameters vs Arguments

These two words are often used interchangeably, but they mean slightly different things. A **parameter** is the name listed inside the function definition (a placeholder). An **argument** is the actual value you pass in when calling the function. In `def add(a, b):`, a and b are parameters; in `add(3, 5)`, 3 and 5 are arguments.

3.1 Positional Arguments

Positional arguments are matched to parameters purely by their **order** — the first value you pass fills the first parameter, the second value fills the second parameter, and so on.

CODE

```
# values are matched to parameters by position
def add(a, b):
    print(a + b)

add(3, 5)
add(10, 20)
```

OUTPUT

```
8
30
```

3.2 Default Arguments

A default argument provides a fallback value that is used automatically **only if no value is passed** for that parameter when the function is called. This makes certain arguments optional.

CODE

```
# A default value is used if no argument is passed
def greet(name="Guest"):
    print("Hello,", name)

greet()
greet("Aisha")
```

OUTPUT

```
Hello, Guest
Hello, Aisha
```

3.3 Keyword Arguments

Keyword arguments are passed using `parameter_name=value`. Because you explicitly name which parameter each value belongs to, the **order no longer matters**.

CODE

```
# Arguments passed by parameter name, order does not matter
def profile(name, age):
    print(name, "is", age, "years old")

profile(age=21, name="Rohan")
```

OUTPUT

```
Rohan is 21 years old
```

Argument Type	Meaning	Example
Positional	Matched to parameters in order	`add(3, 5)`
Default	Used automatically if no value is passed	`def greet(name="Guest"):`
Keyword	Passed using `parameter_name=value`	`profile(age=21, name="A")`

⚠ Watch the order of parameters. Positional arguments must come before any keyword arguments in a function call, otherwise Python raises a **SyntaxError**. For example, `add(a=3, 5)` is invalid, but `add(5, a=3)`... would still conflict if a is set twice — Python raises a **TypeError** in that case instead.

4 RETURN — Sending Data Back From a Function

return vs print

`print()` only displays a value on the screen — it does not give the value back to the program. `return` sends a value back to wherever the function was called, so it can be stored in a variable and used later in your code. If a function has no `return` statement, Python automatically returns the special value `None`.

Example 1 — Using return

CODE

```
def square(n):
    return n * n

result = square(6)
print("The result is:", result)
```

OUTPUT

```
The result is: 36
```

Example 2 — return Stops the Function Immediately

CODE

```
def check_even(n):
    if n % 2 == 0:
        return "Even"
    return "Odd"

print(check_even(10))
print(check_even(7))
```

OUTPUT

```
Even
Odd
```

Returning Multiple Values

A single `return` statement can send back more than one value at once, separated by commas. Python packs them together as a **tuple**, which can be unpacked into separate variables when the function is called.

CODE

```
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([4, 9, 2, 7])
print("Lowest:", low, "Highest:", high)
```

OUTPUT

```
Lowest: 2 Highest: 9
```

print()	return
Displays a value on the screen	Sends a value back to the caller
The value is lost once shown	The value can be stored and reused
Function keeps running after it	Function execution ends immediately
Used mainly for showing output	Used for calculations and logic

5 VARIABLE SCOPE — Local vs Global

Scope determines where in your program a variable can be accessed. Understanding scope is essential before building the ATM Menu project, since it explains why `global` is needed to update the shared `balance` variable.

Local Variables

A variable created **inside** a function only exists inside that function. It is called a local variable, and it disappears once the function finishes running — it cannot be accessed from outside.

Global Variables

A variable created **outside** any function, at the top level of the program, is a global variable. It can be read from anywhere in the program, including inside functions. However, by default, a function **cannot modify** a global variable — assigning to it inside a function just creates a new local variable with the same name instead.

The global Keyword

To let a function actually change a global variable (not just read it), you must declare it with the `global` keyword at the top of the function body. This tells Python: "don't create a new local variable — use the one that already exists outside."

CODE

```
counter = 0  # global variable

def increment():
    global counter
    counter += 1

increment()
increment()
print("Counter is:", counter)
```

OUTPUT

```
Counter is: 2
```

Aspect	Local Variable	Global Variable
Created	Inside a function	Outside any function
Accessible from	Only within that function	Anywhere in the program
Can be modified in a function?	Yes, directly	Only with the `global` keyword
Lifetime	Destroyed when function ends	Exists for the whole program run

⚠ Use `global` sparingly — relying on too many global variables makes programs harder to debug. The ATM Menu below uses it deliberately because the balance genuinely needs to be shared across multiple functions.

6 PRACTICAL EXAMPLE — ATM Menu

Task: Build a simple ATM Menu program using functions for each operation — checking balance, depositing money, and withdrawing money — controlled by a while loop menu.

ATM Menu Program

CODE

```
balance = 1000

def check_balance():
    print("Your balance is:", balance)

def deposit(amount):
    global balance
    balance = balance + amount
    print("Deposited:", amount)

def withdraw(amount):
    global balance
    if amount > balance:
        print("Insufficient funds!")
    else:
        balance = balance - amount
        print("Withdrawn:", amount)

while True:
    print("\n1. Check Balance 2. Deposit 3. Withdraw 4. Exit")
    choice = input("Choose an option: ")

    if choice == "1":
        check_balance()
    elif choice == "2":
        deposit(int(input("Enter amount: ")))
    elif choice == "3":
        withdraw(int(input("Enter amount: ")))
    elif choice == "4":
        print("Thank you for banking with us!")
        break
    else:
        print("Invalid option, try again.")
```

OUTPUT

```
1. Check Balance 2. Deposit 3. Withdraw 4. Exit
Choose an option: 1
Your balance is: 1000
```

Breaking Down the ATM Logic

- Each menu action lives in its own function — this keeps the code organized and reusable
- The `while True` loop keeps showing the menu until the user chooses to exit

- `global balance` lets `deposit()` and `withdraw()` update the shared balance variable
- `break` is used inside the Exit option to stop the loop cleanly
- The built-in functions `print()`, `input()`, and `int()` all work together with the user-defined functions
- This pattern — function per action + menu loop — is the foundation of most real console programs

7 COMMON MISTAKES TO AVOID

Mistake	Wrong	Correct
Forgetting `return`	`def add(a, b): print(a + b)`	`def add(a, b): return a + b` — if you need the value later
Confusing print and return	`result = print(square(4))` → result is `None`	`result = square(4)` — then `print(result)` if needed
Wrong number of arguments	`def add(a, b): ...` then calling `add(5)`	Pass exactly as many arguments as parameters defined
Forgetting `global` for shared variables	`balance = balance - amount` (inside a function, no global)	Add `global balance` at the top of the function body
Calling a function before defining it	`greet()` written above `def greet():`	Always define the function before it is called
Overwriting a built-in name	`list = [1, 2, 3]`	Avoid naming variables after built-ins like `list`, `len`, `str`

8 QUICK REFERENCE GUIDE

Concept	Syntax	Example
Define a Function	`def name():`	`def greet():`
Call a Function	`name()`	`greet()`
Built-in Function	Ready to use, no `def` needed	`len("hi")`, `print()`
User-Defined Function	Written with `def`	`check_balance()`
Positional Argument	`def f(a, b):`	`add(3, 5)`
Default Argument	`def f(a=value):`	`def greet(name="Guest"):`
Keyword Argument	`f(param=value)`	`profile(age=21, name="A")`
Return a Value	`return value`	`return n * n`
Local Variable	Created inside a function	Exists only in that function
Shared/Global Variable	`global name`	`global balance`
Menu Pattern	`while True: ... break`	ATM Menu program

Key Takeaways

- ✓ A function is a reusable, named block of code defined with `def` and run by calling its name
- ✓ Built-in functions come ready-made with Python; user-defined functions are ones you write yourself
- ✓ Arguments let you pass different data into a function each time it runs
- ✓ Default arguments provide a fallback value; keyword arguments are passed by name
- ✓ `return` sends a value back for reuse; `print` only displays it once
- ✓ Local variables live only inside their function; global variables need the `global` keyword to be modified
- ✓ Grouping related actions into functions is the key to building larger, organized programs like the ATM Menu
- ✓ Functions are the foundation for the next topics: modules, file handling, and object-oriented programming

Next Steps

- Practice: complete the 20 function practice questions for Day 4
- Build: write the ATM Menu program (balance, deposit, withdraw), then add a PIN check
- Experiment: rewrite one function to use a default argument, and another to use return instead of print

- Explore: try out 5 more built-in functions you haven't used before (e.g. `sorted()`, `abs()`, `round()`)
- Upload: save today's function programs on GitHub

⚠ **Focus for today:** understand how data flows into and out of a function, and the difference between using a ready-made built-in function versus writing your own. For every function you write, be able to explain what goes in (arguments), what happens inside, and what comes out (return). That mental model is what makes the next topics — modules, file handling, and OOP — click faster.